Library Management Database Portfolio

DESIGN, IMPLEMENTATION AND EVALUATION | 13 NOVEMBER 2026

Organisation: MIVA Library Service - hypothetical academic case study.
Student / ID / course: intentionally omitted for private completion.
Repository: github.com/hacking-specialist/advanced-database-portfolio

1. Scope and relational design

The prototype manages books, physical copies, borrowers, staff, loans, returns and fines. It uses synthetic records and preserves the student-supplied seven-table ER diagram. Books and copies are separate because different physical items of one title have separate lending histories.

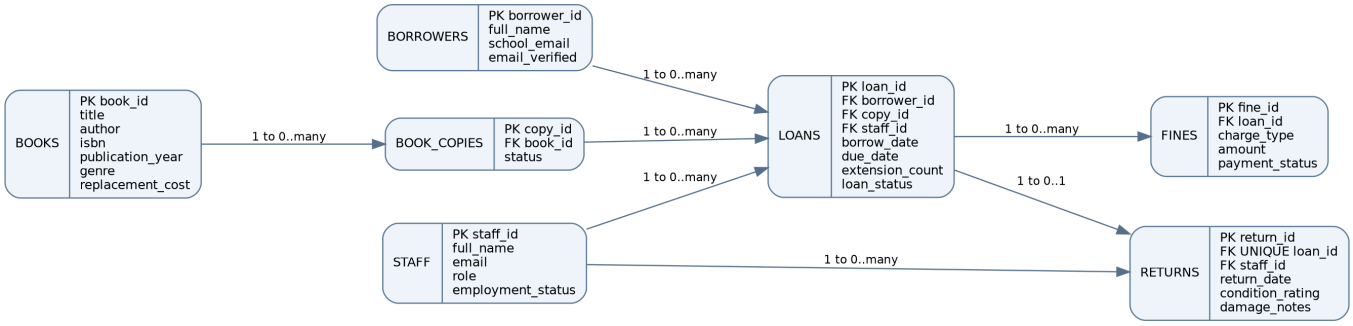


Figure 1. Original student-supplied ER diagram, preserved unchanged.

PostgreSQL 18.6 implements generated primary keys, foreign keys, unique emails and checked dates/amounts. A unique return loan reference permits at most one return per loan. A partial unique index allows historical loans for a copy but only one active loan. The documentation explains functional dependencies through 3NF under the chosen single-author and single-genre interpretation.

2. Implemented workflows and validation

Atomic functions issue, extend and return loans. Assumed policies include verified borrower emails, active staff, 14-day default loans, at most two extensions and NGN 100 per overdue day assessed at return. These are exercise rules, not verified institutional policies.

Both embedded and native PostgreSQL checks passed: invalid references, dates and amounts were rejected; active-loan uniqueness, return chronology, damage notes, fine calculation and rollback were checked. Seed queries returned seven available copies and zero detected workflow inconsistencies.

Design limit: copy/loan status redundancy is maintained through the functions. Direct database-owner writes can bypass workflow rules. Role hardening, payment auditing and ISBN checksum validation remain future work.

Evidence: portfolio-1-database-design/sql/ and evidence/native-validation.json. The original ERD and data dictionary are included in Portfolio 1.

Queries and concurrency

3. Measured query optimisation

A separate analytical loans table contains 200,000 deterministic rows. Five query variants were measured before/after indexing, with seven recorded EXPLAIN ANALYZE trials per phase. Complete outputs were checked for equality. These are repeated, largely warm-cache measurements on a shared Windows host, including instrumentation overhead; they are not production performance guarantees.

Query	Before median (ms)	After median (ms)
Borrower history	107.235	0.171
Active due-date range	85.061	0.914
Status aggregation	166.072	140.433
EXTRACT year filter	150.081	118.700
Half-open year range	112.610	36.333

Selective borrower and due-date searches changed from sequential to bitmap index scans, although sorting remained. Whole-table aggregation kept a sequential scan; its small timing change is not attributed to indexing. A half-open date predicate enabled an indexed plan while EXTRACT on the column did not. Additional indexes increased storage; write overhead was not measured.

4. Real multi-session transaction experiments

Three native PostgreSQL connections provided sessions A, B and an observer. Commit persisted an issue; rollback restored availability. When two borrowers attempted the same copy, the observer recorded a real lock wait. After A committed, B failed as unavailable and exactly one active loan remained.

Read Committed returned 20,000 then 21,000 after another connection committed an update. Repeatable Read returned 21,000 on both reads despite a later committed change. This demonstrates a stable transaction snapshot; it is not a dirty-read example.

Deliberately writing stale client-computed values lost an increment and produced 22,200. Atomic SQL increments preserved both additions and produced 22,300. Locking, isolation and application update logic therefore address different aspects of correctness.

Evidence: Portfolio 2 contains 70 recorded plans, JSON query results, benchmark metadata and summary.csv. Portfolio 3 contains timestamped SQL/events, observed blocking and assertion results.

Documents and distribution

5. NoSQL implementation

MongoDB 8.0.17 stores a five-title catalogue with embedded author metadata and genre arrays plus copy-ID references. This is a documented document-model extension; the relational ERD is unchanged. Actual CRUD operations passed, negative costs and duplicate ISBNs were rejected, and aggregation returned three Computing titles averaging NGN 19,000 replacement cost. No live PostgreSQL-to-MongoDB synchronisation is claimed.

6. Replication, availability and cloud discussion

Three actual MongoDB processes formed a replica set on one host. A majority-written marker was checked on every member. The primary process was stopped; a different primary was observed after 1,609 ms from the start of that stop operation. A subsequent majority write/read succeeded. This single controlled result is not a recovery SLA, network-partition test or cloud deployment.

Majority acknowledgement does not mean every secondary has already applied a write. Three processes on one computer share the same host-failure boundary. Replication is not a backup. The proposed cloud design separates members across availability zones and adds authentication, TLS, monitoring and tested backups. Sharding is analysed as a possible future choice, not reported as implemented.

7. Technical reflection and submission status

The evidence shows why constraints and workflow logic must be considered together, why query plans are needed to interpret timings, and why availability claims must name the failure being survived. Future improvements include least-privilege roles, payment audit records, skewed workload tests, catalogue synchronisation and backup/restore exercises. The student must personalise the separate individual reflection using work they actually reviewed or performed.

The repository includes scripts, outputs and transcripts; generated summaries are not presented as tool screenshots. The repository was observed as public and must be made private for the course. Continued genuine weekly work is needed to demonstrate three months of maintenance. No backdated activity is claimed.

References and AI-use declaration

Primary references: PostgreSQL 18 documentation on constraints, EXPLAIN, partial indexes, isolation and locking; MongoDB documentation on modelling, validation, elections and read/write concern. Full links are in documentation/references.md.

OpenAI Codex assisted with implementation, synthetic data, experiment execution, analysis and documentation. Actual outputs are retained. AI assistance does not establish the student's personal understanding. Personal details were omitted on request; the individual declaration/reflection must be reviewed before submission.